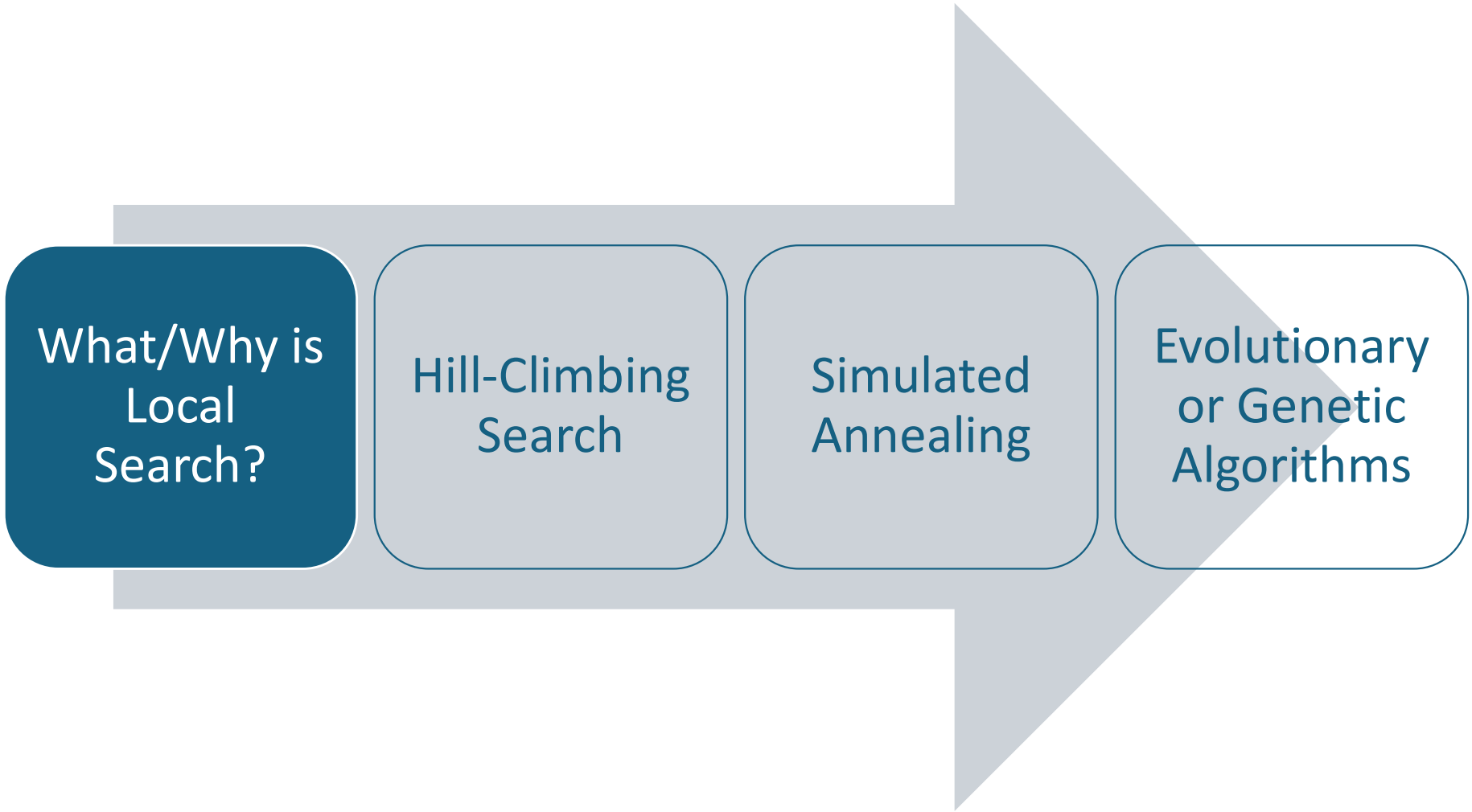
Local Search Algorithms
AIMA Chapters 4.1

Contents


What/Why is Local Search?
Hill-Climbing Search
Simulated Annealing
Evolutionary or Genetic Algorithms

Local Search: Definition

- Local Search is a search technique that starts with one solution and repeatedly moves to a better neighbouring solution until no improvement is possible.
- Imagine you are standing on a hill covered with fog. You cannot see the whole area. You can only see a few meters around you. Your goal is to reach the highest point.
    - Can you see the entire mountain? No
    - Do you know where the highest peak is? No
    - What will you do? "Move towards the higher ground which is visible from current position"
- Conclusion: **This is exactly what Local Search Strategies do -** They make decisions using only the current position and its neighbouring positions.

Why do we even need Local Search (like Hill Climbing, Simulated Annealing, or Genetic Algorithms) when we already have Uninformed Search (BFS/DFS) and Informed Search (A*)?

The Core Difference: Path vs. State

- Systematic Searches (BFS, DFS, UCS, A*):
  - Goal: Find a path from a start state to a goal state.
  - How they work: They build and store a massive tree/graph of paths in memory.

- Local Searches (Hill Climbing, Simulated Annealing, Genetic Algorithms):
  - Goal: Find the best final configuration/state itself.
  - How they work: They start with a complete candidate solution, evaluate how "good" it is, and tweak it locally without keeping track of how they got there.

Why Earlier Algorithms Fail in Optimization Problems

- There are **two main reasons** why BFS, DFS, and A* fail or are completely impractical for optimization problems:

- **Reason A:** Memory Explosion $O(b^d)$ or $O(b^m)$
  - Systematic algorithms must store the search frontier (all open paths) in memory. If a problem has millions of possible choices at every step, A* or BFS will crash your system due to running out of RAM long before finding a solution.
  - Local search algorithms only keep one current state in memory (or a small fixed population in the case of genetic algorithms), using O(1) constant memory.

- **Reason B:** The Path Is Irrelevant.
  - In many real-world engineering and configuration problems, you don't care *how* you reached the final state; you only care about the *final configuration*.

Concrete Example: The N-Queens Problem (e.g., 8-Queens or 100-Queens)

- The Objective: Place N queens on an N X N chessboard so that no two queens attack each other (no two in the same row, column, or diagonal).

An 8-Queens Configuration Example:

```
. . . Q . . . .
. . . . . Q . .
. . Q . . . . .
. . . . . . Q .
Q . . . . . . .
. . . . Q . . .
. Q . . . . . .
. . . . . . . Q
```

How an Informed Search (like A*) tries to solve 100-Queens:

- Starts with an empty board.
- Step 1: Place Queen 1 in Row 1.
- Step 2: Place Queen 2 in Row 2...
- The Problem: It generates a massive search tree tracking path histories like [Q1 at (1,1) -> Q2 at (2,3) -> ...].
- For 100 Queens, the number of possible states to store in the tree is 10^{115}!. A* will instantly run out of memory.

- **How a Local Search (like Hill Climbing) solves 100-Queens:**
- Initial State: Randomly place all 100 queens on the board (one per column) immediately.
- Evaluation: Count how many pairs of queens are attacking each other (e.g., h = 17 attacks).
- Local Action: Move one queen in its column to reduce the number of attacks (e.g., down to h = 12).
- Memory Used: Just the current board state! It throws away the memory of where the queens were a second ago.
- Repeat until h = 0 (a valid solution where no queens attack each other).

Other Real-World Examples Where Local Search is Mandatory

Integrated Circuit (VLSI) Layout:

- Placing millions of transistors on a tiny microchip to minimize wire lengths and heat buildup.
- Path doesn't matter: You don't care about the order in which you placed the components on the blueprint—only the final physical layout matters.

Automated Timetabling / Scheduling:

- Assigning 5,000 university students and 200 professors to rooms without time conflicts.
- Local Search: Start with a random schedule, then swap two classes at a time to reduce room conflicts.

Traveling Salesperson Problem (TSP) with 10,000 Cities:

- A* cannot handle 10,000 cities due to memory limits.
- Genetic Algorithm / Simulated Annealing: Starts with a complete random tour of all 10,000 cities, then randomly swaps edges/routes to iteratively shrink the overall journey distance.

Summary Comparison

Feature	Uninformed / Informed Search (BFS, DFS, UCS, GBFS, A*)	Local Search (Hill Climbing, Annealing, GA)
Primary Goal	Find the optimal path to the goal	Find the optimal final state/configuration
Memory Requirement	High (O(b^d)) — keeps frontier/trees in RAM	Extremely Low (O(1)) — stores only current state(s)
Search Space	Systematic, explores step-by-step from start	Evaluates complete candidate states, tweaks them
Best Used For	Route planning, mazes, puzzle solving	Microchip design, scheduling, 1,000-city TSP, structural optimization

Hill-Climbing Search
aka Greedy Local Search

Idea: keep a single “current” state and try to find better neighboring states.

The Core Algorithm Concept

The basic hill-climbing loop works as follows:

1. Initialize: Start with a complete candidate state (either randomly generated or user-defined).

2. Evaluate: Calculate the state's quality using an **objective function** f(s) (if maximizing) or a **heuristic cost function** h(s) (if minimizing).

3. Inspect Neighbors: Generate adjacent states (successors) produced by making a small, single-step modification to the current state.

4. Step or Terminate:

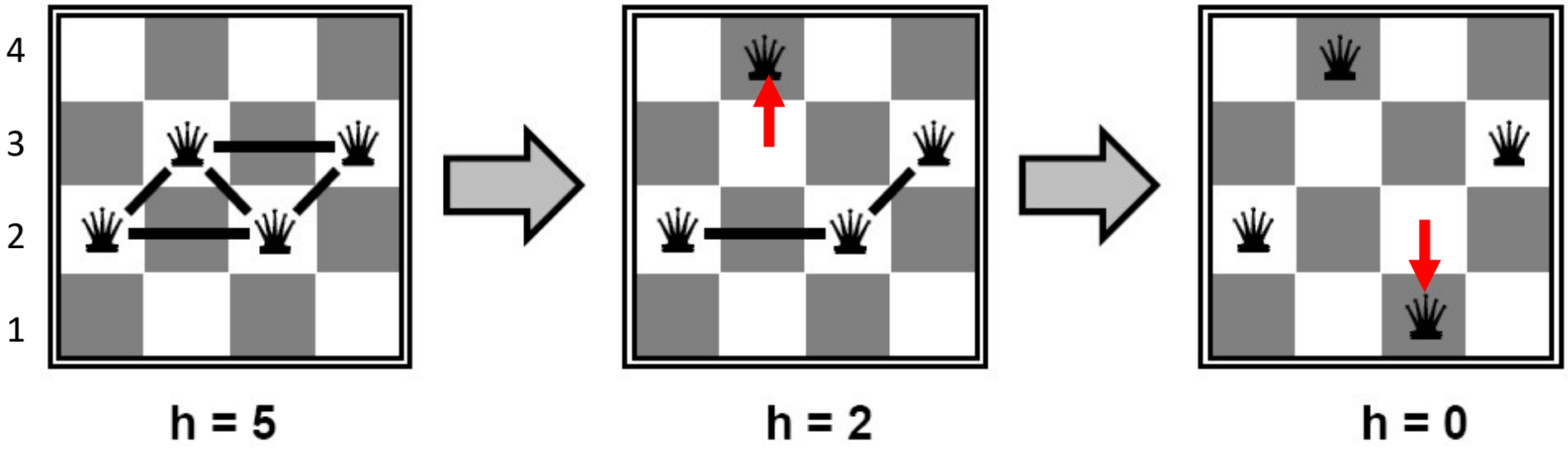
- If a neighboring state has a better value than the current state, move to that neighbor.
- If no neighbor is better than the current state, **stop and return the current state as the solution**.

Example: *n*-Queens Problem

- **Goal:** Put *n* queens on an *n* × *n* board with no two queens on the same row, column, or diagonal.
- **State space:** all possible *n*-queen configurations.
- **State representation:** row position of each queen in its column (e.g., 2, 3, 2, 3)
- **Objective function:** minimize the number of pairwise conflicts.
- **Local neighborhood:** Move one queen anywhere in its column.

Improvement strategy

- Find a local neighboring state (move one queen within its column) to reduce conflicts.

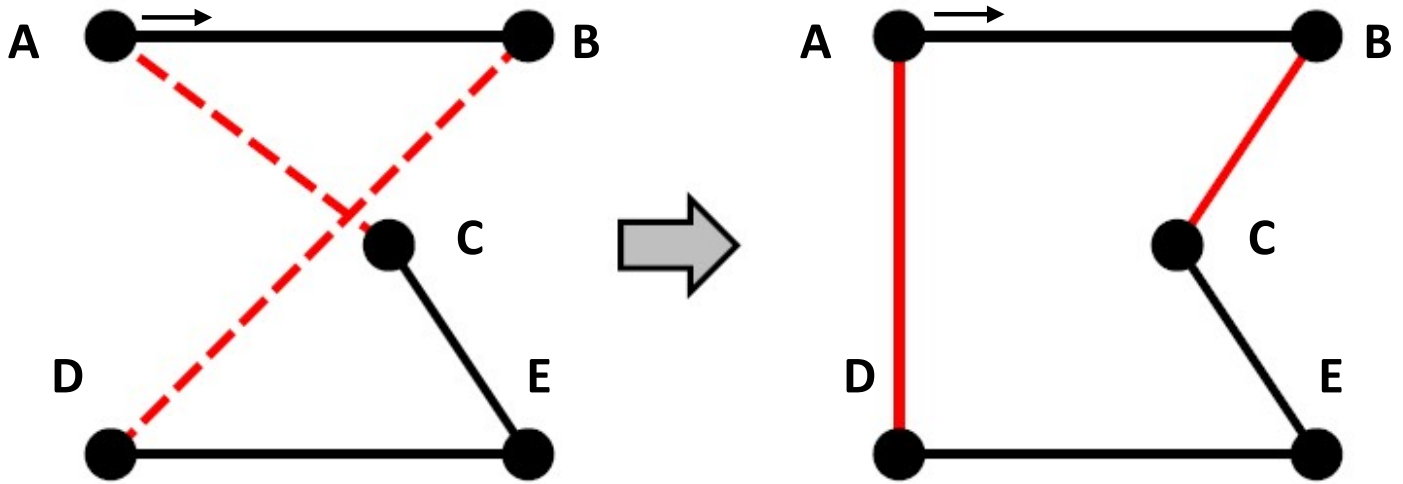


Example: Traveling Salesman Problem 2



- **Goal:** Find the shortest tour connecting n cities
- **State space:** all possible tours
- **State representation:** tour (order in which to visit the cities) = a permutation. There are $n!$ Many permutations.
- **Objective function:** length of tour
- **Local neighborhood:** reverse the order of visiting a few cities

Local move to reverse the order of cities C, E and D:



State representation (permutation): **ABDEC** **ABCED**

Hill-Climbing Search (Greedy Local Search)

Maximization

```
function HILL-CLIMBING(problem) returns a state that is a local maximum
    current ← problem.INITIAL
    while true do
        neighbor ← a highest-valued successor state of current
        if VALUE(neighbor) ≤ VALUE(current) then return current
        current ← neighbor
```

We often start with a random state

Use lowest-valued successor and $\geq$ for minimization

Variants (define what successor to choose)

a. **Steepest ascent hill climbing:** Check all possible successors and choose the highest-valued successor.

b. **Stochastic hill climbing:** Choose randomly among **all** uphill (improvement) moves.

c. **First-choice stochastic hill climbing**: Generate randomly **one** new successor at a time and only move to better ones. This is what people often mean by "stochastic hill climbing." It is equivalent to b, but computationally much cheaper (does not evaluate h for each neighbor).

Minimization



$h = 17$

Major Variations of Hill Climbing

Variant 1: Simple Hill Climbing (First-Choice Hill Climbing)

- Strategy: Generates and evaluates successor states **one by one** in a fixed or random order. The algorithm immediately selects the **very first neighbor** that provides an improvement over the current state and steps to it.
- **Pros:** Extremely fast per step; does not waste time evaluating all possible neighbors.
- **Cons:** Dependent on the order in which neighbors are generated; may take a erratic path toward peaks.
- **Example:**
    - *Current State:* h(s) = 17.
    - *Generated Neighbor 1:* h(s_1) = 18 (Worse -> Rejected).
    - *Generated Neighbor 2:* h(s_2) = 14 (Better -> **Accepted immediately**, skipping evaluation of remaining ALL neighbors!).

Variant 2: Steepest-Ascent Hill Climbing (Greedy Hill Climbing)

- Strategy: Generates **all** possible successor states from the current state, evaluates every single one, and moves to the neighbor that offers the **greatest improvement** (steepest gradient).
- **Pros:** Makes the most aggressive, direct local progress per step.
- **Cons:** Computationally expensive when the number of neighbors is large.
- **Example:**
    - *Current State:* h(s) = 17.
    - All other possible neighbors are evaluated.
    - Neighbor values range from 12 to 21.
    - *Selection:* The algorithm chooses the neighbor with h(s) = 12 because it yields the largest decrease in attacks.

Variant 3: Stochastic Hill Climbing

- Strategy: Chooses **randomly** among all uphill (improving) neighbors. The probability of selecting a particular neighbor can be uniform or weighted proportionally to the steepness of improvement.
- **Pros:** Adds randomness to help avoid getting funneled into narrow local slopes.
- **Cons:** Converges more slowly than steepest ascent on simple landscapes.
- **Example:**
    - *Current State:* h(s) = 17.
    - Out of ALL neighbors, 5 neighbors improve h(s) (values: 12, 14, 15, 15, 16).
    - Rather than strictly picking 12, it assigns a higher selection probability to 12 and lower to 16, then picks one at random.

Variant 4: Random-Restart Hill Climbing

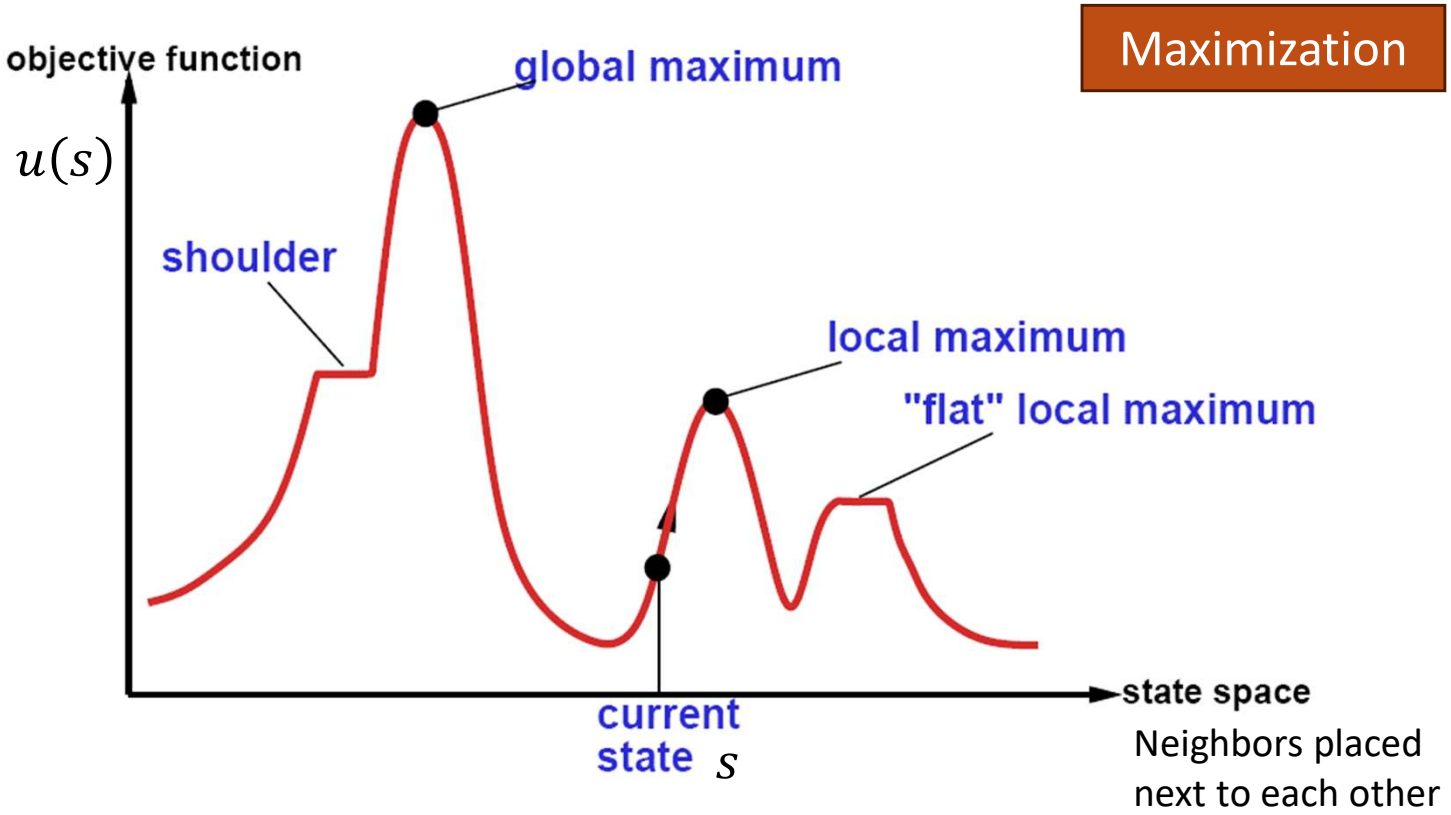
- Strategy: Conducts a series of independent hill-climbing searches. Each iteration starts from a **randomly generated initial state**. It saves the best result found across all runs and terminates when a global optimum is reached or a maximum restart budget is exhausted.
- **Pros:** Simple, complete, and overcomes local traps given enough restarts.
- **Example:**
    - *Run 1:* Starts at random state -> gets stuck at local minimum h(s) = 3.
    - *Run 2:* Starts at new random state-> gets stuck at local minimum h(s) = 1.
    - *Run 3:* Starts at new random state -> reaches global minimum h(s) = 0 (**Success!**).

Problems & Traps in Hill Climbing

- Because hill climbing only makes greedy, strictly improving moves, it frequently gets trapped in non-optimal states on complex landscapes:

- **Local Maxima (or Minima):**
  - A peak that is higher than all adjacent neighbors, but lower than the global peak. Because all local steps go downward, standard hill climbing halts and falsely reports success or failure.
- **Plateaus (and Shoulders):**
  - A flat area of the search space where neighboring states have the exact same value. The algorithm has no directional gradient to choose a direction.
  - *Solution:* Allow **sideways moves** (stepping to neighbors with equal value) with a limit (e.g., max 100 consecutive sideways steps) to traverse plateaus.
- **Ridges:**
  - A sequence of local maxima joining together along a narrow slope. Because individual single-variable modifications move orthogonally to the ridge direction, every single step appears to go downhill, trapping the algorithm even though a diagonal path upward exists.

The State Space "Landscape"



How to escape local maxima?

→ Random restart hill-climbing can help.

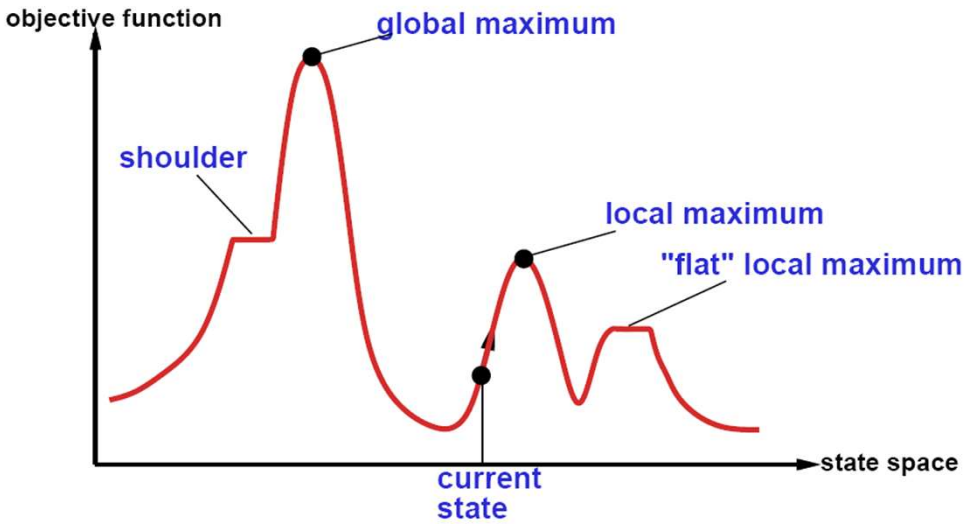
What about "shoulders" (called "ridges" in higher-dimensional spaces)?

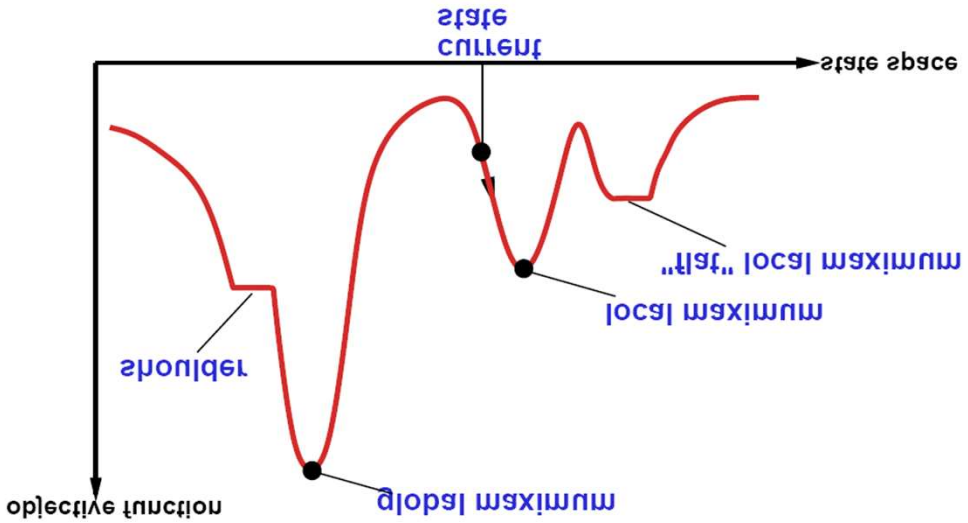
→ Hill-climbing that allows sideways moves and uses momentum.

Minimization vs. Maximization

- The name **hill climbing** used in AI implies **maximizing a utility function**.
- Optimizers often prefer to state problems as **minimization** problems to reduce **cost or loss** and refer to the same type of algorithm as **gradient descent**.
- Minimization and maximization are equivalent problems:

$$\max\big(f(x)\big) \quad \Leftrightarrow \quad \min\big(-f(x)\big)$$





Simulated Annealing

Idea: Using heat to escape local optima…

Motivation

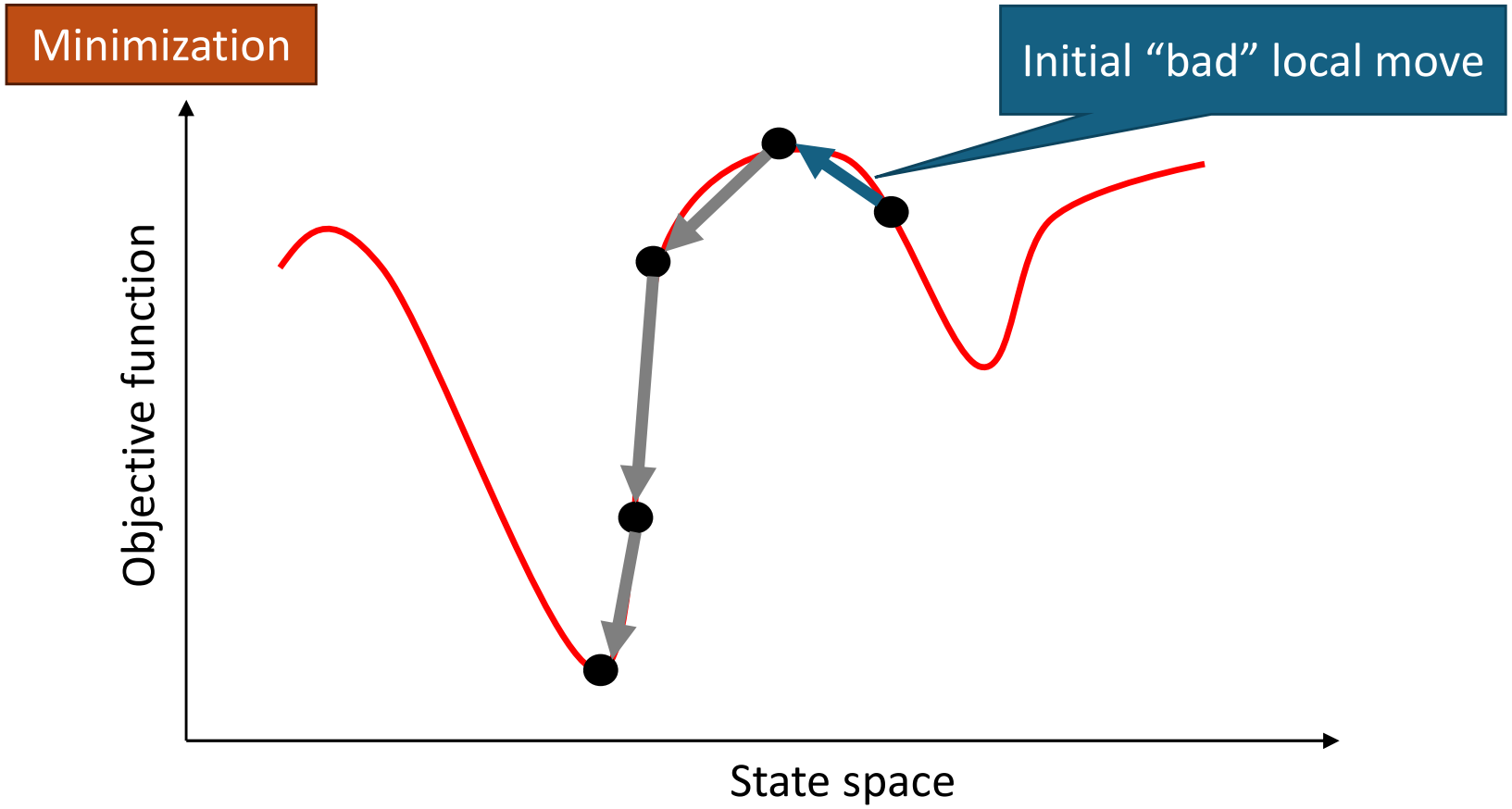
- **"How do blacksmiths make strong steel?"**
- The answer: They **heat the metal to a very high temperature** and then **cool it slowly**.
- Why?
- At high temperatures: Metal atoms move freely. They can escape poor arrangements. They explore many configurations.
- As the temperature decreases: Movement becomes smaller.
- Eventually atoms settle into a stable arrangement with **minimum energy**. This process is called **annealing**.

Main Idea

- Instead of always choosing a better state like Hill Climbing, Simulated Annealing sometimes says:
- **"I'll accept a worse move if the temperature is high."**
- Why?
- Because that worse move may help escape a local optimum.

Idea of Simulated Annealing

- Use first-choice stochastic hill climbing + escape local minima by **allowing some "bad" moves** but gradually decreasing their frequency.
- Inspired by the process of controlled cooling of glass or metals. Decreasing the temperature means decreasing the chance of accepting bad moves.



Key Idea of Simulated Annealing

Unlike Standard Hill Climbing (which strictly moves better states), Simulated Annealing allows **bad moves (uphill moves)** with a probability P:

$$P = e^{-\frac{\Delta E}{T}}$$

- $\Delta E = h(\text{neighbor}) - h(\text{current})$: The increase in conflict count ($\Delta E > 0$ for a worse neighbor).
- T**(Temperature)**: Starts high and slowly decreases (cools down) over time.
  - **High** T**(Hot)**: High probability of accepting worse moves (exploring).
  - **Low** T**(Cold)**: Low probability of accepting worse moves (exploiting, behaves like standard Hill Climbing).

How temperature affects probability

Suppose the new solution is **5 units worse** ($\Delta E = 5$).

Temperature (T)	Probability (P=e^{-5/T})	Interpretation
100	0.951	Almost always accept
10	0.607	Often accept
5	0.368	Sometimes accept
2	0.082	Rarely accept
1	0.007	Almost never accept

Why does this happen?

High temperature (beginning)

- The algorithm explores freely. Even bad moves have a high chance of being accepted. Helps escape local maxima/minima.

Example:

- Current cost = 20 , New cost = 25 , ΔE = 5 , T = 100

$$P = e^{-5/100} = e^{-0.05} = 0.951$$

There is a **95.1% chance** of accepting the worse solution.

Low temperature (towards the end)

- The algorithm becomes conservative. It mostly accepts only better solutions.

Example: ΔE = 5 , T = 1, $P = e^{-5} = 0.0067$

Only about a **0.67% chance** of accepting the worse solution.

Think of temperature as **the courage to take risks**:

- **High temperature** → "I'm willing to try risky moves because I might find a much better solution later."
- **Low temperature** → "I'm close to the optimum, so I'll avoid risky moves."
- This gradual reduction in risk-taking is exactly what makes simulated annealing effective.

How is this used in the algorithm?

1. Choose an initial solution S.

2. Set an initial temperature T.

3. Repeat until stopping condition:

a. Generate a neighboring solution S'.

b. If S' is better:

Accept S'.

Else:

Compute P = e^(-ΔE/T).

Generate a random number r in [0,1].

If r < P:

Accept S'.

Else:

Reject S'.

c. Reduce the temperature.

$T = \alpha \times T$ $(0 < \alpha < 1)$

4. Return the best solution found.

Why do we need a random number?

- The **random number** is what makes **Simulated Annealing probabilistic instead of deterministic**.
- Suppose the acceptance probability for a worse solution is: $P = 0.60$
- This does **not** mean "accept exactly 60% of the move."
- Instead, it means **there is a 60% chance of accepting it**.

- The probability tells us how likely a worse solution should be accepted; the random number decides whether it is accepted in this particular trial.

Why not simply accept whenever $P > 0.5$?

If we did that, the algorithm would behave **deterministically**.

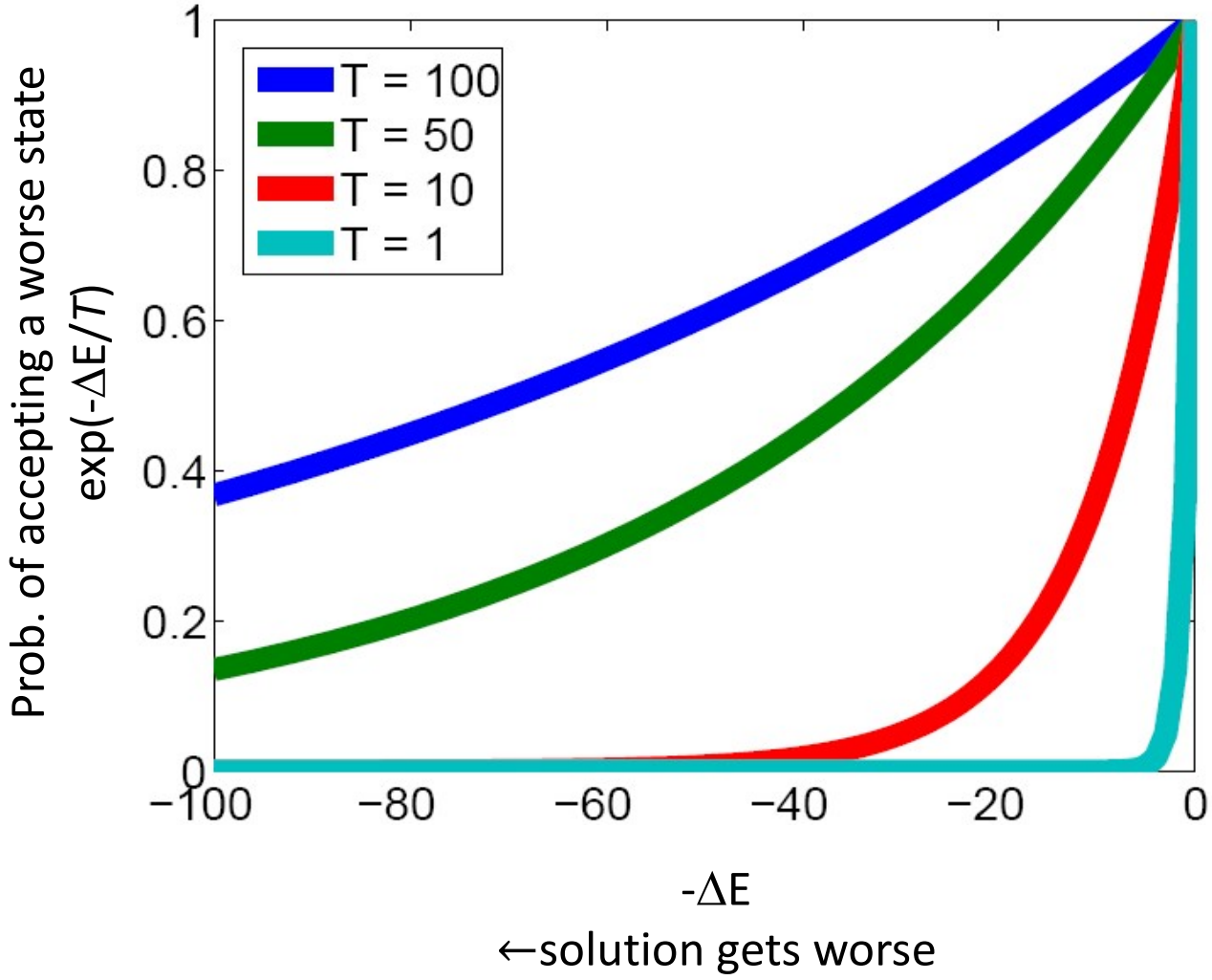
- For example, if $P = 0.6$: It would **always** accept.
- If $P = 0.4$: It would **never** accept.
- This is **not** what simulated annealing is designed to do.

Using a random number ensures that:

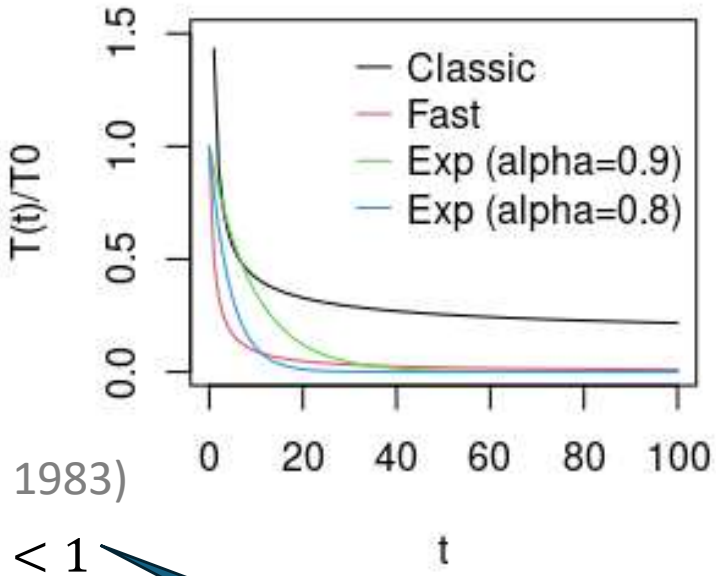
- A move with $P = 0.8$is accepted about **80% of the time**.
- A move with $P = 0.2$is accepted about **20% of the time**.

The Effect of Temperature

Convert the changes due to “bad” moves into an acceptance probability depending on the temperature. The criterion uses the negative part of the exponential function.



Cooling Schedule



The cooling schedule is very important.
Schedules for the temperature at time t:

- **Classic simulated annealing:** $T_t = T_0 \frac{1}{\log(1+t)}$
- **Exponential cooling** (Kirkpatrick, Gelatt and Vecchi; 1983)

$$T_t = T_0 \alpha^t \quad \text{for} \quad 0.8 < \alpha < 1$$

- **Fast simulated annealing** (Szy and Hartley; 1987)

$$T_t = T_0 \frac{1}{1+t}$$

This is the most popular choice!

Notes:

- Choose T_0 to provide a high probability $p_0 = e^{-\frac{\Delta E}{T_0}}$ that an average move will be accepted at time $t = 0$. For p_0 0.8 or 0.9 is often used. A conservative choice for ΔE is to use the worst possible move. This is a rule of thumb!
- Stopping rule: T_t will not become 0 but very small. Stop when $T < \epsilon$ (ϵ is a very small constant).
- The best schedule is typically determined by trial-and-error. The goal is to have a low chance of getting stuck in a local optima.

Simulated Annealing Search

Guarantee

If the temperature is decreased **slowly enough** (using the classic cooling schedule), then simulated annealing search will find a **global optimum** with a probability approaching one.

However, this schedule takes too long and thus is **impractical**.

Practical solution

- Accept suboptimal results and determine the best cooling schedule and local move need experimentally.

Evolutionary Algorithms / Genetic Algorithms

A Population-based Metaheuristics

Introduction analogy 1

Imagine a forest with 100 deer.

- Some deer can run very fast.
- Some deer are slow.

Every day, lions hunt deer.

- Fast deer escape.
- Slow deer are caught.

After many years,

- Mostly fast deer survive.
- Fast deer produce babies.
- Those babies also inherit the "fast running" characteristic.

Generation after generation,

the average speed of deer increases.

This process is called

- **Natural Selection**
- **Survival of the Fittest**

Nature continuously improves species without anyone designing them.

Analogy 2

Suppose there are mango trees. Some trees produce

- sweeter mangoes
- bigger mangoes
- disease-resistant mangoes

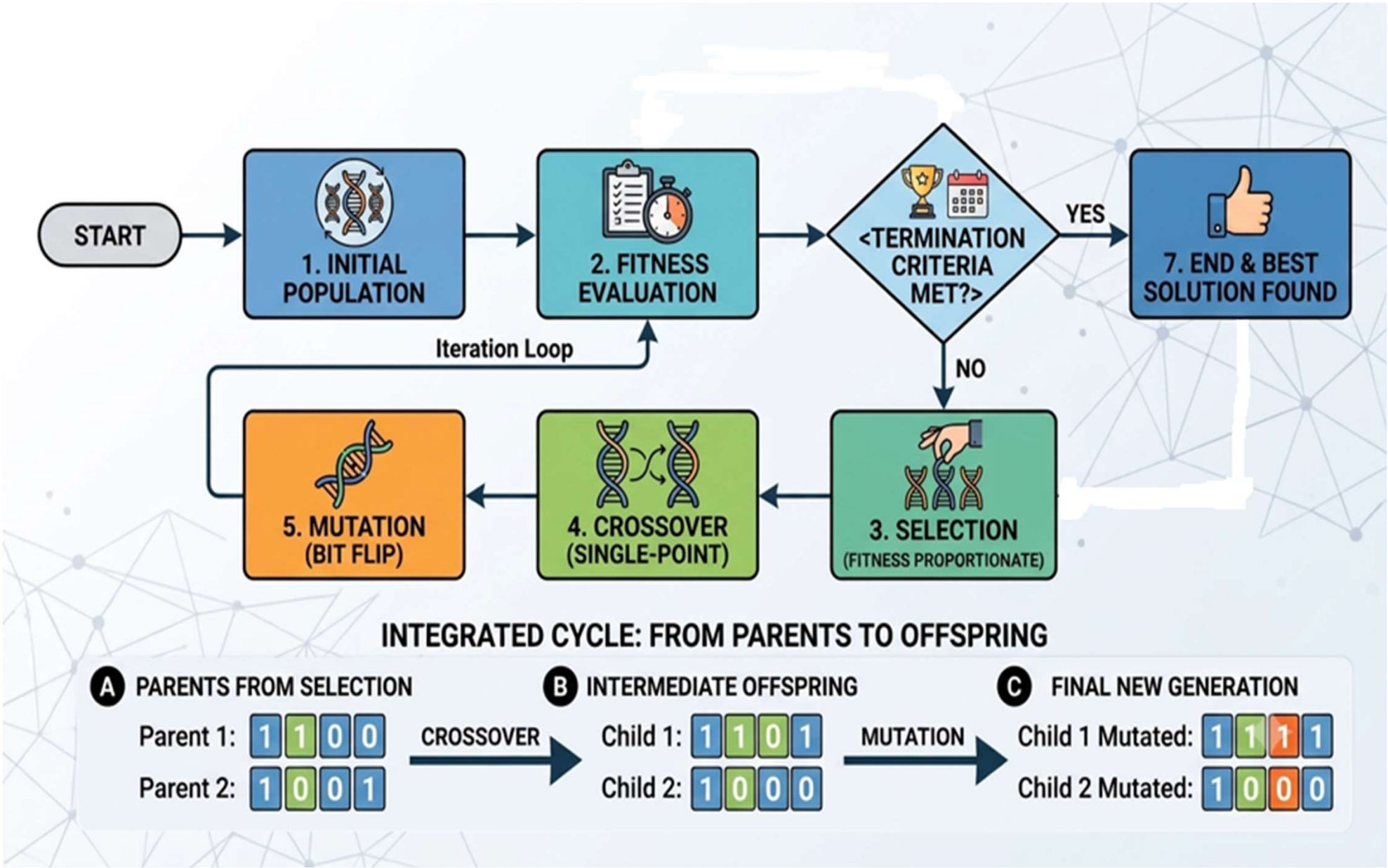
- Farmers usually choose seeds from the best trees.
- Next generation again has better fruits.
- After several generations,
- the overall quality improves.
- No one designed the perfect mango.
- It evolved.

- **Relate it to Computer Science**
- **"Can we make computers solve problems the same way nature improves living organisms?"**
- Scientists answered **YES!**
- **Genetic Algorithm is an optimization technique that searches for the best solution by imitating the process of biological evolution.**

Basic Idea

Instead of searching one solution,
GA starts with many random solutions.
Each solution is evaluated.
Good solutions survive. (Selection)
Bad solutions disappear.
Good solutions combine together. (Crossover)
Small random changes are introduced. (Mutation)
After several generations,
better and better solutions emerge.

The Genetic Algorithm Lifecycle Flow Chart



Summary

- "Nature has been solving optimization problems for millions of years through evolution. Genetic Algorithms borrow the same principles—selection, reproduction (crossover), and mutation—to help computers discover high-quality solutions to complex problems.“
- **Genetic Algorithm:** Keep many solutions simultaneously and let them evolve like living organisms.

Problem Statement – Example of GA working

- Suppose we want to find the **largest 4-bit binary number**.
- Possible numbers are:

  0000 = 0

  0001 = 1

  0010 = 2

  …

  1111 = 15
- The best answer is obviously: 1111
- But imagine **we don't know that**.
- Our job is to discover it using a Genetic Algorithm.

Step 1: Setup & Initial Population

- We define our representation and start with a random initial population of 4 individuals ($N = 4$).
- **Chromosome Representation:** 4-bit binary string (e.g., 1010).
- **Fitness Function:** $f(x) =$ Decimal Value of x.
- Let's pick an initial population randomly:

A_1: 0101 (Decimal: 5)

A_2: 1001 (Decimal: 9)

A_3: 0010 (Decimal: 2)

A_4: 1100 (Decimal: 12)

Step 2: Fitness Evaluation

Calculate the fitness $f(x)$for each individual and its selection probability $P(i) = \frac{f(i)}{\sum f}$:

Individual	**Binary String**	**Fitness f(x)**	**Selection Probability P(i)**
A_1	0101	5	$5/28 \approx 18\%$
A_2	1001	9	$9/28 \approx 32\%$
A_3	0010	2	$2/28 \approx 7\%$
A_4	1100	12	$12/28 \approx 43\%$
Total		**28**	**100%**

Step 3: Selection (Roulette Wheel Method)

We select 4 parents based on their fitness probability. Fitter individuals like $A_4(12)$ have a much higher chance of being picked than weak ones like $A_3(2)$.

- Picked Parent 1: $A_4(1100)$
- Picked Parent 2: $A_2(1001)$
- Picked Parent 3: $A_4(1100)$
- Picked Parent 4: $A_1(0101)$

(Notice A_3 was discarded due to low fitness, while A_4 was chosen twice.)

Step 4: Crossover (Recombination)

We pair the parents and apply **Single-Point Crossover** at position 2 (after the 2nd bit):

- **Pair 1 (A_4 and A_2):**
  Parent 1: 11 | 00
  Parent 2: 10 | 01
  Child 1: 1101 (Decimal: 13)
  Child 2: 1000 (Decimal: 8)
- **Pair 2 (A_4 and A_1):**
  Parent 3: 11 | 00
  Parent 4: 01 | 01
  Child 3: 1101 (Decimal: 13)
  Child 4: 0100 (Decimal: 4)

Step 5: Mutation

We introduce a slight random bit flip with a small mutation rate (e.g., flipping 1 bit across the entire offspring pool).

- Take **Child 1** (1101) and flip the 4th bit: 1101 $\rightarrow$**1111**
- Other children remain unchanged: 1000, 1101, 0100.

New Population Evaluation (Generation 1 Results)

Let's look at our new population:

- **Initial Average Fitness:** $\frac{28}{4} = 7.0$
- **New Average Fitness:** $\frac{15+8+13+4}{4} = 10.0$

Through selection, crossover, and mutation, the algorithm discovered the optimal solution 1111 in just 1 generation!

New Individual	Binary	Decimal Fitness
N_1	1111	**15**
N_2	1000	8
N_3	1101	13
N_4	0100	4

IMPORTANT NOTE: If the optimal solution not reached in generation 1 then the same process will be repeated for generation 2, and so on.

Selection Algorithms

- Selection means choosing parents from the current population to produce the next generation.

Individual	Binary String	Decimal Value	Fitness f(x)
A_1	0101	5	5
A_2	1001	9	9
A_3	0010	2	2
A_4	1100	12	12
Total			**28**

To understand how **Roulette Wheel**, **Tournament**, and **Rank Selection** differ, let's apply each method directly to our earlier 4-bit population ($N = 4$).

1. Roulette Wheel Selection (Fitness-Proportionate)

Each individual gets a slice of a wheel proportional to its raw fitness score. Total wheel size = 28.

- **Calculate Slices (Probabilities):**
  - A_1: $5/28 \approx 17.9\%$
  - A_2: $9/28 \approx 32.1\%$
  - A_3: $2/28 \approx 7.1\%$
  - A_4: $12/28 \approx 42.9\%$
- **How Selection Works:** Spin the wheel 4 times.
  - Because A_4owns $42.9\%$of the wheel, a random spin is most likely to land on it.
  - Because A_3owns only 7.1%, it rarely gets selected.
- **Resulting Parent Set (Example):** $[A_4, A_2, A_4, A_1]$

2. Tournament Selection

- Instead of looking at the entire population at once, we pick small groups at random and let them "compete."
- **Set Tournament Size (k):** Let's choose $k = 2$.
- **Run 4 Tournaments to pick 4 parents:**
  - **Tournament 1:** Draw A_1(0101, fit: 5) and A_4(1100, fit: 12).
    - *Winner:* A_4(12 beats 5).
  - **Tournament 2:** Draw A_2(1001, fit: 9) and A_3(0010, fit: 2).
    - *Winner:* A_2(9 beats 2).
  - **Tournament 3:** Draw A_3(0010, fit: 2) and A_4(1100, fit: 12).
    - *Winner:* A_4(12 beats 2).
  - **Tournament 4:** Draw A_1(0101, fit: 5) and A_2(1001, fit: 9).
    - *Winner:* A_2(9 beats 5).
- **Resulting Parent Set:** $[A_4, A_2, A_4, A_2]$

3. Rank Selection

Raw fitness scores are ignored. Individuals are sorted by fitness and assigned a **Rank** from 1(worst) to N(best). Probabilities are calculated based on these ranks.

Individual	**Fitness f(x)**	**Rank (R)**	**Rank-Based Probability P(R)=∑RR**
A_3	2	**1** (Worst)	$1/10 = 10\%$
A_1	5	**2**	$2/10 = 20\%$
A_2	9	**3**	$3/10 = 30\%$
A_4	12	**4** (Best)	$4/10 = 40\%$
Total Rank Sum		**10**	**100%**

- **How Selection Works:** We spin a wheel based on these fixed **Rank Probabilities** instead of raw values.
- **Why use this over Roulette?**
  - Suppose A_4 had a huge fitness of **1,000** while A_1 had **5**. In Roulette selection, A_4 would take 99% of the wheel and eliminate all other choices instantly.
  - In **Rank Selection**, A_4 still gets Rank 4 (40% probability), preserving equal chance and population diversity regardless of scale.
- **Resulting Parent Set (Example):** $[A_4, A_2, A_3, A_1]$

Side-by-Side Comparison on the 4-Bit Example

Selection Method	**Basis for Selection**	**Key Feature in This Example**
Roulette Wheel	Direct Fitness Ratio	A_4gets $42.9\%$of the selection pool due to its raw score $(12/28)$.
Tournament ($k = 2$)	Local Head-to-Head	A_3lost every match because its score (2) was lower than any randomly drawn competitor.
Rank	Sorted Position ($1 \rightarrow 4$)	A_4gets 40%chance $(4/10)$ based on being 1st place, ignoring raw gap sizes.

Crossover Methods

In Genetic Algorithms, **Crossover (Recombination)** is the operator used to combine the genetic information of two parent chromosomes to generate new offspring. It simulates sexual reproduction, allowing beneficial traits from both parents to merge.

Using our **4-bit binary representation**, let's look at two selected parent chromosomes:

- **Parent 1 (P_1):** 1 1 0 0 (Decimal: 12)
- **Parent 2 (P_2):** 1 0 0 1 (Decimal: 9)

1. Single-Point Crossover

A single crossover point along the chromosome is chosen at random. The binary strings are split at this point, and the tail segments are swapped between the parents.

- **Selected Cut Point:** Position 2 (between the 2nd and 3rd bit)
  - P_1: 1 1 **|** 0 0
  - P_2: 1 0 **|** 0 1
- **Swapping Tails:**
  - **Child 1 (C_1):** 1 1 + 0 1 →**1 1 0 1** (Decimal: 13)
  - **Child 2 (C_2):** 1 0 + 0 0 →**1 0 0 0** (Decimal: 8)
- **Key Characteristic:** Simple to implement, but can break up building blocks of genes that are located far apart.

2. Two-Point Crossover

Two crossover points are chosen at random. The gene segment falling **between** these two points is swapped between the parents.

- **Selected Cut Points:** Position 1 and Position 3 (swapping only the middle bits)
  - P_1: 1 **|** 1 0 **|** 0
  - P_2: 1 **|** 0 0 **|** 1
- **Swapping Middle Segment:**
  - **Child 1 (C_1):** 1 + 0 0 + 0 →**1 0 0 0** (Decimal: 8)
  - **Child 2 (C_2):** 1 + 1 0 + 1 →**1 1 0 1** (Decimal: 13)
- **Key Characteristic:** Treats the chromosome as a ring structure. It is better at preserving groups of traits regardless of where they sit in the binary string.

Mutation Methods

In Genetic Algorithms, **Mutation** is a background operator used to maintain genetic diversity from one generation to the next. It alters one or more gene values in a chromosome from its initial state, preventing the algorithm from getting stuck in local optima.

Let's explore the main mutation techniques using our **4-bit binary representation** and permutation/sequence formats where applicable.

1. Bit Flip Mutation

This is the standard mutation operator used for binary-coded chromosomes (0s and 1s). Individual bits are selected based on a small mutation probability (P_m) and inverted (0 becomes 1, and 1 becomes 0).

- **Original Chromosome:** 1 1 0 0 (Decimal: 12)
- **Selected Target:** Bit position 3 (the 3rd bit 0)
- **Action:** Flip 0 $\rightarrow$1
- **Mutated Chromosome: 1 1 1 0** (Decimal: 14)
- **Best for:** Binary representations.

2. Swap Mutation

- Two random positions in the chromosome are selected, and their gene values are interchanged. In a binary string, swapping two *identical* bits results in no change, so it is most impactful when the bits differ.
- **Original Chromosome:** 1 1 0 0
- **Selected Positions:** Position 2 (1) and Position 4 (0)
- **Action:** Swap values at index 2 and index 4
- **Mutated Chromosome: 1 0 0 1** (Decimal: 9)
- **Best for:** Permutation-based problems (e.g., Traveling Salesperson Problem) where every gene value must remain unique.

3. Scramble Mutation

A contiguous subset (sub-string) of genes is chosen at random, and the values within that selected window are shuffled randomly.

- **Original Chromosome:** 1 1 0 0
- **Selected Subset Range:** Positions 2 through 4 ([1, 0, 0])
- **Action:** Randomly shuffle the bits inside the selected range (e.g., [1, 0, 0] →[0, 0, 1])
- **Mutated Chromosome: 1 0 0 1** (Decimal: 9)
- **Best for:** Sequence/order-based problems where local gene order needs to be re-explored.

4. Inversion Mutation

A contiguous subset of genes is chosen at random, and the entire sequence within that range is completely flipped (reversed).

- **Original Chromosome:** 1 1 0 0
- **Selected Subset Range:** Positions 1 through 3 ([1, 1, 0])
- **Action:** Reverse the sequence [1, 1, 0] $\rightarrow$[0, 1, 1]
- **Mutated Chromosome: 0 1 1 0** (Decimal: 6)
- **Best for:** Permutation and ordering problems (e.g., routing algorithms), as it alters adjacency relationships while preserving the set of genes.

Summary Comparison

Mutation Type	Target Mechanism	Example Transformation (1 1 0 0)	Primary Use Case
Bit Flip	Inverts bit value ($0 \leftrightarrow 1$)	1 1 0 0 →**1 1 1 0**	Binary Chromosomes
Swap	Swaps two gene positions	1 1 0 0 →**1 0 0 1**	Permutations & Sequences
Scramble	Shuffles a random sub-segment	1 1 0 0 →**1 0 0 1**	Ordering / Routing Problems
Inversion	Reverses a random sub-segment	1 1 0 0 →**0 1 1 0**	Traveling Salesperson Problem